
Getting Real with JUnit

In the last chapter we wrote tests against a simple example class named `ScoreCollection` that calculates an arithmetic mean. Working through that exercise helped us get a good handle on JUnit fundamentals.

Pat wasn't impressed, however. "All that effort to test a tiny class that averages a bunch of numbers? Real code isn't so simple."

True, Pat, although the previous chapter was more about getting you comfortable with JUnit. We could write more tests against `ScoreCollection`, but it's time to move on to testing code closer to the reality of the average system.

In this chapter we'll spend a little time looking at a meatier bit of code that we want to test. The analysis effort will help us to focus on writing a test that covers one path through the code. We'll then write a second test to verify a second path through the code. Our second effort will demonstrate how things get easier after we've tackled the first test.

We'll also increase our focus on test structure. We'll delve deeper into the arrange-act-assert (AAA) mnemonic for test layout, as well as the `@Before` annotation, which allows putting common initialization code in one place.

Understanding What We're Testing: The Profile Class

We'll be writing tests against portions of an application named `iloveyouboss`, a job-search website designed to compete with sites like `Indeed` and `Monster`. It takes a different approach and attempts to match prospective employees with potential employers, and vice versa, much as a dating site would. Employers and employees both create profiles by answering a series of multiple-choice or yes-no questions. The site scores profiles based on criteria from the other party and shows the best potential matches from the perspective of both employee and employer.

The authors reserve the right to monetize the site, make a fortune, retire, and do nothing but support the kind readers of this book.

Let's look at a core class in `iloveyouboss`, the `Profile` class:

```

iloveyouboss/6/src/iloveyouboss/Profile.java
Line 1 package iloveyouboss;
-
- import java.util.*;
-
5 public class Profile {
-     private Map<String, Answer> answers = new HashMap<>();
-     private int score;
-     private String name;
-
10    public Profile(String name) {
-        this.name = name;
-    }
-
-     public String getName() {
15         return name;
-     }
-
-     public void add(Answer answer) {
-         answers.put(answer.getQuestionText(), answer);
20    }
-
-     public boolean matches(Criteria criteria) {
-         score = 0;
-
25         boolean kill = false;
-         boolean anyMatches = false;
-         for (Criterion criterion: criteria) {
-             Answer answer = answers.get(
-                 criterion.getAnswer().getQuestionText());
-
30             boolean match =
-                 criterion.getWeight() == Weight.DontCare ||
-                 answer.match(criterion.getAnswer());
-
-             if (!match && criterion.getWeight() == Weight.MustMatch) {
35                 kill = true;
-             }
-             if (match) {
-                 score += criterion.getWeight().getValue();
-             }
40             anyMatches |= match;
-         }
-         if (kill)
-             return false;
-         return anyMatches;
45    }

```

```

-
-     public int score() {
-         return score;
-     }
50 }

```

This looks like the code we come across often! Let's walk through it.

A Profile (line 5) captures answers to relevant questions one might ask about a company or a job seeker. For example, a company might ask of a job seeker, "Are you willing to relocate?" A Profile for a job seeker might contain an Answer object with the value true for that question. You add Answer objects to a Profile by using the add() method (line 18). A Question contains the text of a question plus the allowable range of answers (true or false for yes/no questions). The Answer object references the corresponding Question and contains an appropriate value for the answer (line 29).

A Criteria instance (see line 22) is simply a container that holds a bunch of Criteria. A Criterion (first referenced on line 27) represents what an employer seeks in an employee, or vice versa. It encapsulates an Answer object and a Weight object, which represents how important the right answer to a question is.

The matches() method takes a Criteria object (line 22) and iterates through each Criterion (line 27) in an effort to determine whether or not the criteria are a match for the answers in the profile (line 30). If any criterion is weighted as an absolute must but doesn't match the corresponding profile answer, then matches() returns false (lines 34 and 42). If no criteria match corresponding answers in the profile, matches() also returns false (lines 26, 40, and 44). In all other cases matches() returns true.

The matches() method also has a side effect: when a criterion matches the corresponding profile answer, the score for the profile is increased by the weighted value of the criteria (line 37).

All that sounds logical, but the matches() method is reasonably involved, and we want to know if it works as expected. Let's figure out how to write a test against it.

Determining What Tests We Can Write

You could write dozens or even hundreds of tests against any method of reasonable complexity. You want to think instead about how many tests you *should* write. You can look at branches and potentially impactful data variants in the code. A starting point is to look at loops, if statements, and complex

conditionals. After that, consider data variants: what happens if a value is null or zero? How do the data values affect evaluation of the conditionals in the code?

Beyond a simple happy path where the Criteria instance holds a single matching Criterion object, each of the following conditions merits consideration for affecting an existing test case or introducing another test case:

- The Criteria instance holds no Criterion objects (line 27).
- The Criteria instance holds many Criterion objects (line 27).
- The Answer returned from answers.get() is null (line 29).
- Either of criterion.getAnswer() or criterion.getAnswer().getQuestionText() returns null (line 29).
- match resolves to true because criterion.getWeight() returns Weight.DontCare (line 30).
- match resolves to true because value matches criterion.getValue() (line 30).
- match resolves to false because both conditions return false (line 30).
- kill gets set to true because match is false and criterion.getWeight() equals Weight.MustMatch (line 34).
- kill does not get changed because match is true (line 34).
- kill does not get changed because criterion.getWeight() is something other than Weight.MustMatch (line 34).
- score gets updated because match is true (line 37).
- score does not get updated because match is false (line 37).
- The matches method returns false because kill is true (line 42).
- The matches method returns true because kill is false and anyMatches is true (lines 42 and 44).
- The matches method returns false because kill is false and anyMatches is false (lines 42 and 44).

This list of fifteen conditions (and we could probably come up with a few more good ones) is based on a surface reading of the code. All we're doing so far is figuring how the code can branch or how data variants can cause different things to happen. When we get down to writing tests, we'll have to better understand what the code really does.

We'd likely end up writing fewer than fifteen tests, however. Some of these conditions only have relevance if other conditions are met, so we'd combine those dependent conditions into a single test. But the key point remains: to comprehensively test `matches()`, we would need to write a good number of tests.

We'll instead triage a bit better. We wrote the code (well, let's assume you helped, which means we'll have to remind you of what you wrote), so we probably have a good idea where the most interesting and thus risky areas lie. In a similar vein, when we examine our freshly written code to write tests, we recognize that it has meaty parts that concern us most.

Covering One Path

The bulk of the “interesting” logic in `matches()` resides in the body of the `for` loop. Let's write a simple test that covers one path through the loop.

Two points that glancing at the code should make obvious: we need a `Profile` instance, and we need a `Criteria` object to pass as an argument to `matches()`.

By analyzing the code in `matches()` and looking at the constructors for `Criteria`, `Criterion`, and `Question`, we figure out how to piece together a useful `Criteria` object.

The analysis lets you write this part of the *arrange* portion of the test:

```
iloveyouboss/6/test/iloveyouboss/ProfileTest.java
@Test
public void test() {
    Profile profile = new Profile("Bull Hockey, Inc.");
    Question question = new BooleanQuestion(1, "Got bonuses?");
    Criteria criteria = new Criteria();
    Answer criteriaAnswer = new Answer(question, Bool.TRUE);
    Criterion criterion = new Criterion(criteriaAnswer, Weight.MustMatch);
    criteria.add(criterion);
}
```

(From here on out, we're expecting you to code along with us. We won't be as explicit. If you see a new code snippet, figure that you'll need to make some changes on your end.)

Paraphrased in brief: after creating a profile, create a question (*Got bonuses?* They'd better!). The next three lines are responsible for putting together a `Criterion`, which is an answer plus a weighting of the significance of that answer. The answer, in turn, is a question and the desired value (`Bool.TRUE`) for the answer to that question. Finally, the criterion is added to a `Criteria` object.

(In case you're wondering, the `Bool` class is a wrapper around an enum that has values 0 and 1. We don't claim that the code we're testing is good code.)

In `matches()`, for each `Criterion` object iterated over in the `for` loop, the code retrieves the corresponding `Answer` object in the `answers` `HashMap` (line 29). That means you must add an appropriate `Answer` to the `Profile` object:

```
iloveyouboss/7/test/iloveyouboss/ProfileTest.java
```

```
@Test
public void test() {
    Profile profile = new Profile("Bull Hockey, Inc.");
    Question question = new BooleanQuestion(1, "Got bonuses?");
    Answer profileAnswer = new Answer(question, Bool.FALSE);
    profile.add(profileAnswer);
    Criteria criteria = new Criteria();
    Answer criteriaAnswer = new Answer(question, Bool.TRUE);
    Criterion criterion = new Criterion(criteriaAnswer, Weight.MustMatch);
    criteria.add(criterion);
}
```

We finish up the test by acting and asserting. We also change its name to aptly describe the scenario it demonstrates:

```
iloveyouboss/8/test/iloveyouboss/ProfileTest.java
```

```
@Test
public void matchAnswersFalseWhenMustMatchCriteriaNotMet() {
    Profile profile = new Profile("Bull Hockey, Inc.");
    Question question = new BooleanQuestion(1, "Got bonuses?");
    Answer profileAnswer = new Answer(question, Bool.FALSE);
    profile.add(profileAnswer);
    Criteria criteria = new Criteria();
    Answer criteriaAnswer = new Answer(question, Bool.TRUE);
    Criterion criterion = new Criterion(criteriaAnswer, Weight.MustMatch);
    criteria.add(criterion);

    boolean matches = profile.matches(criteria);
    assertFalse(matches);
}
```

We were able to piece together a test based on our knowledge of the `matches` method, verifying that one pathway through the code works as (apparently) intended. If neither of us knew much about the code, we would have had to spend a bit more time carefully reading through the code to understand what it does, building up more and more of a real test as we went.

Think about maintaining our test. It's ten lines of code, which doesn't seem like much. But if we write tests to cover all fifteen conditions described previously, it seems like it could get out of hand. Fifteen tests times ten lines each sounds like a lot to maintain for a target method of fewer than twenty lines.

From a cognitive standpoint, the ten lines require careful reading, particularly for someone else who knows nothing about the code we wrote.

Tackling a Second Test

Let's write a second test to see if our concerns are warranted. Taking a look at the assignment to the `match` local variable (starting at line 30 in `Profile.java`), it appears that `match` gets set to `true` when the criterion weight is `DontCare`. Code in the remainder of the method suggests that `matches()` should return `true` if a sole criterion sets `match` to `true`.

Each unit test in JUnit requires its own context: JUnit doesn't run tests in any easily determinable order, so we can't have one test depend on the results of another. Further, JUnit creates a new instance of the `ProfileTest` class for each of its two test methods.

We must then make sure our second test, `matchAnswersTrueForAnyDontCareCriteria`, similarly creates a `Profile` object, a `Question` object, and so on:

```
iloveyouboss/9/test/iloveyouboss/ProfileTest.java
```

```
@Test
public void matchAnswersTrueForAnyDontCareCriteria() {
    Profile profile = new Profile("Bull Hockey, Inc.");
    Question question = new BooleanQuestion(1, "Got milk?");
    Answer profileAnswer = new Answer(question, Bool.FALSE);
    profile.add(profileAnswer);
    Criteria criteria = new Criteria();
    Answer criteriaAnswer = new Answer(question, Bool.TRUE);
    ➤ Criterion criterion = new Criterion(criteriaAnswer, Weight.DontCare);
    criteria.add(criterion);

    boolean matches = profile.matches(criteria);
    ➤ assertTrue(matches);
}
```

The second test looks darn similar to `matchAnswersFalseWhenMustMatchCriteriaNotMet`. In fact, the two highlighted lines are the only real difference between the two tests. Maybe we can reduce the 150 potential lines of test code by eliminating some of the redundancy across tests. Let's do a bit of *refactoring*.

Initializing Tests with @Before Methods

The first thing to look at is common *initialization* code in all (both) of the tests in `ProfileTest`. If both tests have such duplicate logic, move it into an `@Before` method. For each test JUnit runs, it first executes code in any methods marked with the `@Before` annotation.

The tests in `ProfileTest` each require the existence of an initialized `Profile` object and a new `Question` object. Move that initialization to an `@Before` method named `create()` (or `bozo()` if you want to irritate your teammates—the name is arbitrary).

```
iloveyouboss/10/test/iloveyouboss/ProfileTest.java
```

```
public class ProfileTest {
    > private Profile profile;
    > private BooleanQuestion question;
    > private Criteria criteria;
    >
    > @Before
    > public void create() {
    >     profile = new Profile("Bull Hockey, Inc.");
    >     question = new BooleanQuestion(1, "Got bonuses?");
    >     criteria = new Criteria();
    > }

    @Test
    public void matchAnswersFalseWhenMustMatchCriteriaNotMet() {
        Answer profileAnswer = new Answer(question, Bool.FALSE);
        profile.add(profileAnswer);
        Answer criteriaAnswer = new Answer(question, Bool.TRUE);
        Criterion criterion = new Criterion(criteriaAnswer, Weight.MustMatch);
        criteria.add(criterion);

        boolean matches = profile.matches(criteria);

        assertFalse(matches);
    }

    @Test
    public void matchAnswersTrueForAnyDontCareCriteria() {
        Answer profileAnswer = new Answer(question, Bool.FALSE);
        profile.add(profileAnswer);
        Answer criteriaAnswer = new Answer(question, Bool.TRUE);
        Criterion criterion = new Criterion(criteriaAnswer, Weight.DontCare);
        criteria.add(criterion);

        boolean matches = profile.matches(criteria);

        assertTrue(matches);
    }
}
```

The initialization lines moved to `@Before` disappear from each of the two tests, making them a little bit easier to read.

Imagining that JUnit chooses to run `matchAnswersTrueForAnyDontCareCriteria` first, here's the sequence of events:

1. JUnit creates a new instance of `ProfileTest`, which includes the uninitialized `profile`, `question`, and `criteria` fields.
2. JUnit calls the `@Before` method, which initializes each of `profile`, `question`, and `criteria` to appropriate instances.

3. JUnit calls `matchAnswersTrueForAnyDontCareCriteria`, executing each of its statements and marking the test as passed or failed.
4. JUnit creates a new instance of `ProfileTest`, because it has another test to process.
5. JUnit calls the `@Before` method for the new instance, which again initializes fields.
6. JUnit calls the other test, `matchAnswersFalseWhenMustMatchCriteriaNotMet`.

If you don't believe that JUnit creates a new instance for each test it runs, crank up your debugger or drop in a few `System.out.println` calls. JUnit works that way to force the issue of independent unit tests. If both `ProfileTest` tests ran in the same instance, you'd have to worry about cleaning up the state of the shared `Profile` object.

You want to minimize the impact any one test has on another (which means you also want to avoid static fields in your test classes). Imagine you have several thousand unit tests, with numerous interdependencies among tests. If test *xyz* fails, your effort to determine why increases dramatically, because you must now look at all the tests that run before *xyz*.

Our tests read a bit better now, but let's make another pass at cleaning them up. We *inline* some local variables, creating a more condensed yet slightly more readable arrange portion of each test:

```
iloveyouboss/11/test/iloveyouboss/ProfileTest.java
```

```
@Test
public void matchAnswersFalseWhenMustMatchCriteriaNotMet() {
    profile.add(new Answer(question, Bool.FALSE));
    criteria.add(
        new Criterion(new Answer(question, Bool.TRUE), Weight.MustMatch));

    boolean matches = profile.matches(criteria);

    assertFalse(matches);
}

@Test
public void matchAnswersTrueForAnyDontCareCriteria() {
    profile.add(new Answer(question, Bool.FALSE));
    criteria.add(
        new Criterion(new Answer(question, Bool.TRUE), Weight.DontCare));

    boolean matches = profile.matches(criteria);

    assertTrue(matches);
}
```

What we like about this version of the two tests is that each of the arrange/act/assert sections is an easily digested line or two. If necessary, we can look at any @Before methods, but we design their contents to put low-information clutter out of sight.

How Ya Feelin' Now?

This book provides two starter examples for a few reasons. The first example ([Chapter 1, *Building Your First JUnit Test*, on page 3](#)) demonstrates the basics of how to use JUnit and minimizes the distraction of involved logic. That's not good enough for Pat, who would claim, "See? Unit testing is only good on toy examples."

Hence the second example in this chapter, which contains a reasonably complex set of logic. We hope it doesn't dissuade you from wanting to unit-test your code, but this is reality: methods like `matches()` embody a surprising number of branches and cases that each suggest the need for yet another test.

So far, we've covered only two paths through the `matches()` method. The smaller effort required to write the second test hopefully makes it apparent that we could write the other tests—up to thirteen more—in reasonable time. But it would still be a bunch more tests.



Clean up your tests regularly to simplify writing more tests.

We'll let you choose whether or not to write those missing tests. It's really not that much more effort, and it pays off by giving you high confidence that the `matches()` method works as expected.

We're going to write those tests so that we have confidence to change the `Profile` code in later chapters in this book. You can take a look at the source distribution to see the tests we write. You'll find the set of tests—seven in all, not so bad—in `iloveyouboss/13/test/iloveyouboss/ProfileTest.java`.

Don't give up on unit testing just yet! There's a better way of structuring code in the first place so that things are a bit simpler and so you don't feel compelled to write as many tests. In [Chapter 9, *Bigger Design Issues*, on page 107](#), you'll see how a better design makes it easier for you to write tests. And in [Chapter 12, *Test-Driven Development*, on page 153](#), you'll see how writing the tests as you build each small bit of code makes writing tests a natural and even fun process.

After

In this chapter you learned enough to start writing tests with JUnit. However, writing good unit tests takes a little more discipline than slapping some asserts around your code. Also, JUnit provides a number of fun little features that we didn't touch on in this example.

In the next chapter you'll learn more about the various types of JUnit assertions that you can use to help verify expected conditions in your tests.